MyCodeSchool

Code review — slot-mate

Flavius · runde de review pe proiectul tău, cea mai nouă sus.

Runda 2 — 2026-11-11 — sha `b5a2408`

Scop: branch-ul `feat/implement-feature-1`, commit-urile `e2d3ca0` „Tests in category" și `b5a2408` „teste si docker": update + delete pe categorii, handler-ul de 404, testele de serviciu, `Dockerfile` și `docker-compose-slot-mate-api.yaml`.

Metodă: `./gradlew clean test` pe o copie a branch-ului, apoi aplicația pornită **cu compose-ul tău** (MySQL 8.4 + imaginea din `Dockerfile`-ul tău) și fiecare endpoint apelat cu `curl`. Codul tău nu a fost atins.

Poarta de compilare: ✅ `main` și testele se compilează · 🔴 `./gradlew test` : **10 din 15 teste pică**, toate cu `404` (B1).

Ce merge: `Dockerfile`-ul (build în două etape) și compose-ul pornesc din prima: MySQL devine `healthy`, aplicația se conectează și ascultă pe 8085. `CategoryNotFoundException` → 404 prin `@RestControllerAdvice` e exact cum trebuia. `POST` cu un nume existent → 409, inclusiv pe alt case (`barber` după `Barber`).

Ce am primit la rulare:

Cerere	Răspuns	Ce trebuia (contractul `slot-mate-docs/openapi.yaml`)
`POST /api/categories` `{"name":"Barber"}`	404	201
`POST /api/categories/add` `{"name":"Barber"}`	201	ruta nu există în contract
`GET /api/categories`	404	200, array
`GET /api/categories/all` pe tabel gol	404	200 `[]`
`GET /api/categories/all` cu 2 categorii	200 `{"categoryResponseList":[...]}`	200 `[...]`, ordonat după `name`
`PUT /update/Barber` `{"name":""}`	200, numele rămâne	în afara contractului (M1)
`PUT /update/Barber` `{"name":"x" × 51}`	500	în afara contractului (M1)

Cerere	Răspuns	Ce trebuia (contractul `slot-mate-docs/openapi.yaml`)
`PUT /update/Barber {"name":"Barber"}`	409	în afara contractului (M1)
`DELETE /delete/Nimic`	404	în afara contractului (M1)

🔴 Critice

B1 — `CategoryController.java:42` : `POST` mutat pe `/add` → 10 teste roșii

Dovada: `./gradlew test` → `Status expected:<201> but was:<404>` (și `<400>` / `<409>` → `<404>`) în `CategoryControllerTest` (7) și `CategoryIntegrationTest` (3). Am pus pe copie `@PostMapping` fără `"/add"` → **15/15 verde**. E singura cauză.

Mecanism: un endpoint e identificat de **metodă + cale**. Testele și contractul sunt clientul: ele trimit `POST /api/categories`, pentru că așa scrie în `openapi.yaml`. Spring caută un handler pentru perechea asta, nu găsește niciunul (al tău ascultă acum pe `/api/categories/add`) și răspunde 404 înainte să ajungă la codul tău. Un test care trecea și acum pică îți spune că ai schimbat comportamentul văzut din afară. Asta e treaba testelor: înainte de push le rulezi pe toate, nu doar pe cele noi.

💡 `./gradlew test` înainte de fiecare commit. Cum arată linia `:42` ca să treacă testele care deja existau? 💡 În REST verbul e metoda HTTP (`POST`), nu un cuvânt din URL (`/add`). Aceeași regulă la `:36`, vezi B2.

B2 — `GET` nu respectă contractul: cale, formă, listă goală

`CategoryController.java:36` · `CategoryQueryServiceImpl.java:17-19` · `CategoryResponseList.java`

Dovada: `GET /api/categories` → 404 (ruta e `/all`); pe tabel gol `/all` → 404 `The category could not be found`; cu date → `{"categoryResponseList":[...]}`. Contractul: `GET /api/categories` → 200, **array**, ordonat după `name`, `[]` când nu e nimic. Acesta era exercițiul tău din runda 1 și a rămas neatins.

Mecanism: în runda 1 lista goală dădea 500. Acum dă 404, pentru că ai adăugat handler-ul pentru `CategoryNotFoundException`, iar `getAllCategories` încă aruncă excepția asta. 404 înseamnă „resursa de la adresa asta nu există". Colecția `/api/categories` există mereu, chiar și goală, la fel ca un dosar gol. Clientul (frontend-ul, aplicația mobilă) face `for` pe lista primită: pe `[]` nu afișează nimic, pe un 404 intră pe ramura de eroare.

💡 Ce întoarce `findAll()` când tabela e goală? Mai ai nevoie de `if` după asta? 💡 Contractul cere un array. Ce tip întoarce metoda din controller ca JSON-ul să înceapă cu `[`? 💡 „Ordonat după `name`": Spring Data știe să ordoneze singur, după numele metodei din repository. 💡 Tot în metoda asta: `findAll()` e apelat de două ori, deci două `SELECT`-uri.

B3 — `CategoryController.java:49` · `CategoryUpdateRequest.java:3-5` : validarea la update

Dovada: `PUT /update/Barber {"name":""}` și `{}` → **200**, numele rămâne `Barber`, iar clientul crede că a reușit. `{"name":"x" × 51}` → **500**, în log `ConstraintViolationException ... Category ... during update time`.

Mecanism: e B2 din runda 1, pe altă metodă. `@Valid` pune Spring să valideze obiectul de lângă el, după adnotările de pe câmpurile lui. Ale tale stau pe: - `@PathVariable String name`: un `String` simplu, fără nicio regulă. N-are ce valida. - `CategoryUpdateRequest`, pe care se află regulile importante: n-are niciun `@NotBlank` sau `@Size`, iar `@Valid` nici nu e pus lângă el.

Așa că nimic nu oprește cererea la intrare. Numele gol trece de `if`-ul de la `CategoryCommandServiceImpl.java:36` și e ignorat în tăcere. Numele prea lung ajunge la Hibernate, care revalidează entitatea la salvare, după `@Size(max = 50)` din `Category`. Atunci e prea târziu pentru un 400, deci iese 500.

💡 Compară `CategoryUpdateRequest` cu `CategoryCreateRequest`. Ce lipsește, și lângă ce parametru se pune `@Valid`? 💡 Dacă scoți update-ul (M1), B3 pleacă odată cu el. Mecanismul îți trebuie însă la PATCH-ul de mai târziu.

🟡 Importante

- **M1 — update și delete nu fac parte din F1:** `CategoryController.java:30–34, :48–52`. Contractul are pentru categorii doar `GET` și `POST`. Pe 2026-10-21 le-am scos intenționat, ca feature-ul să meargă cap-coadă (cod → test → Docker → EC2) cu cât mai puțin cod. Scoate-le din branch: rămân în istoria git și revin când apar în contract, pe `{categoryId}` (UUID), nu pe nume.
- **M2 — testul de delete nu testează ștergerea:** `CategoryCommandServiceImplTest`, `deleteCategoryReturnsTheSavedCategory`. Am șters pe copie linia `categoryRepository.delete(category)`: testul tot trece. Verifică doar valoarea întoarsă, nu efectul. 💡 Cu ce metodă Mockito verifici că `delete` chiar a fost apelat? O folosești deja în `createCategoryDoesNotSaveWhenTheNameExists`.
- **M3 — lipsește testul pentru `GET`:** pleacă odată cu B2. Câte un caz pentru listă goală și pentru ordine.
- **M4 — dubla căutare a rămas:** `CategoryCommandServiceImpl.java:33–35, :44–45`. `findCategoryByName` e apelat de două ori (verifici, apoi cauți iar). Din runda 1, B3. 💡 `orElseThrow(...)`.

🟢 Cleanups

- **C1:** `CategoryCommandServiceImpl.java:3`: `@Transactional` tot din `jakarta.transaction` (runda 1, C4). În Spring folosește `org.springframework.transaction.annotation`.
- **C2:** `CategoryController.java:32`: două `{}` în mesaj, un singur argument.
- **C3:** `CategoryController.java:31`: `@PathVariable()`, fără paranteze. `CategoryController.java:12`: import nefolosit.
- **C4:** `CategoryCommandService.java:10–12`: `throws` pe excepții `RuntimeException` nu e obligatoriu și nu ajută aici.
- **C5:** testele noi: `// Arrange / Act / Assert` lipsește la ultimele două, iar `assertEquals` amestecat cu `assertThat`. Alege unul (restul fișierului folosește `assertThat`).
- **C6:** `Dockerfile:1, :7`: comentarii în cod. Regula proiectului e fără comentarii.
- **C7:** `application.yaml:12`: `dialect: MySQLDialect` se detectează singur (warning la pornire).

Before / After

	Acum	Corect
B1	`@PostMapping("/add")`	`@PostMapping` → `POST /api/categories`
B2	`@GetMapping("/all")` → `{"categoryResponseList": [...]}`, 404 pe gol	`@GetMapping` → `List<CategoryResponse>`, `[]` pe gol, ordonat după `name`

	Acum	Corect
B3	`record CategoryUpdateRequest(String name)`, `@Valid` pe `@PathVariable`	regulile din `CategoryCreateRequest` pe câmp, `@Valid` lângă `@RequestBody`

Ordinea de lucru

1. B1: `./gradlew test` verde (15/15).
2. M1: scoți update/delete, testele lor și `CategoryUpdateRequest`.
3. B2 + M3: `GET` după contract, cu testul lui. Abia atunci F1 e gata de EC2.

Întrebări

1. Ce cod HTTP primește un client care trimite `GET /api/categories` pe o bază fără categorii, și de ce nu e 404?
2. Pe ce obiect se uită Spring când vede `@Valid`, și de ce `@Valid` pe un `String` nu face nimic?
3. Testul tău de delete trece și dacă metoda nu șterge nimic. Ce ar trebui să verifice un test ca să pice atunci când codul e greșit?

Runda 1 — 2026-10-19 — sha `f11db8a`

- ✅ B1 `@GeneratedValue` pe `Category`: închis (UUID, `00efe4c`)
- ✅ B2 `@Valid` la `POST`: închis (`00efe4c`) · 🔴 aceeași regulă la update → runda 2, B3
- ✅ B3 `Optional == null`: închis (`isEmpty()`) · 🟡 dubla căutare a rămas → runda 2, M4
- ✅ B4 excepțiile → 500: închis (handler 404 + 409) · 🔴 lista goală acum 404 → runda 2, B2
- ✅ B5 testul de controller: înlocuit cu `CategoryControllerTest` (`c214baf`)
- 🔴 M1 rutele contractului: NEREZOLVAT, `POST` mutat pe `/add` → runda 2, B1, B2
- 🔴 M2 update/delete în afara F1: readăugate → runda 2, M1
- ✅ M3 `@RestController` · ✅ M4 config dublat (`application.properties` scos, `DB_PASSWORD`)
- ✅ M5 `application-test.yaml` scos · ✅ M6 entitățile scoase din F1 (în istoria git)
- ✅ C1 mapper · ✅ C2 repository · ✅ C3 mesaje în engleză · 🟢 C4 `jakarta.transaction` → runda 2, C1